

Unit 3 – Amortized Analysis

CS 201 - Data Structures II

Spring 2023

Habib University

Syeda Saleha Raza

Asymptotic Analysis - Recap

Complexity Analysis

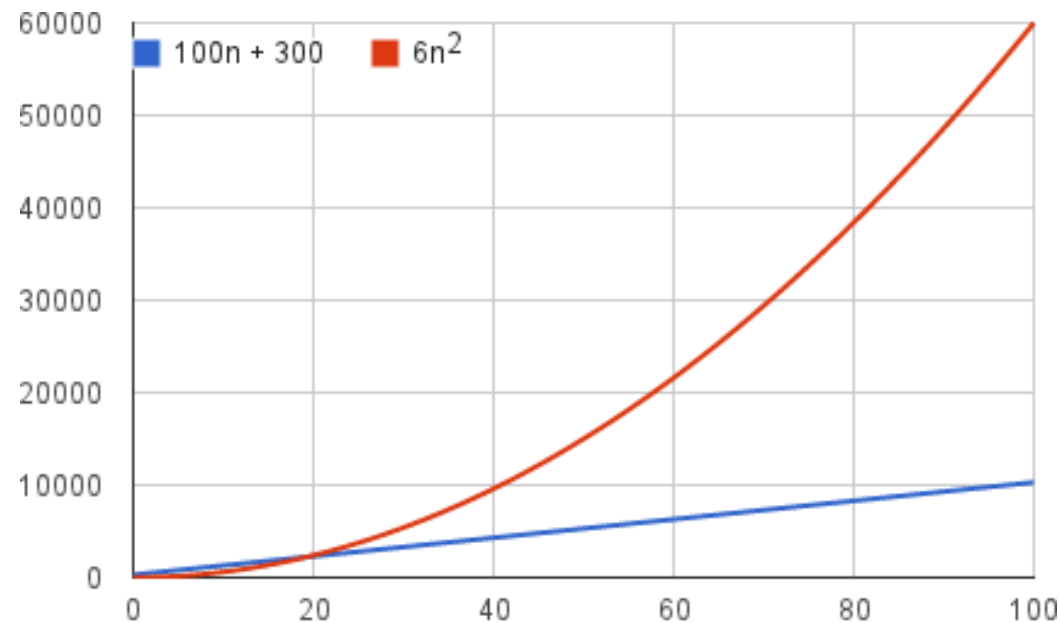
- When we are trying to find the complexity of the function/ procedure/ algorithm/ program, we are not interested in the exact number of operations that are being performed. Instead, we are interested in the relation of the number of operations to the problem size.

Example

```
a=5
b=6
c=10
for i in range(n):
    for j in range(n):
        x = i * i
        y = j * j
        z = i * j
    for k in range(n):
        w = a*k + 45
        v = b*b
d = 33
```

Asymptotic Analysis

For example, suppose that an algorithm, running on an input of size n takes $6n^2 + 100n + 300$. $6n^2 + 100n + 300$
The $6n^2$ term becomes larger than the remaining terms, $100n + 300$ once n becomes large enough, 20 in this case.



- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

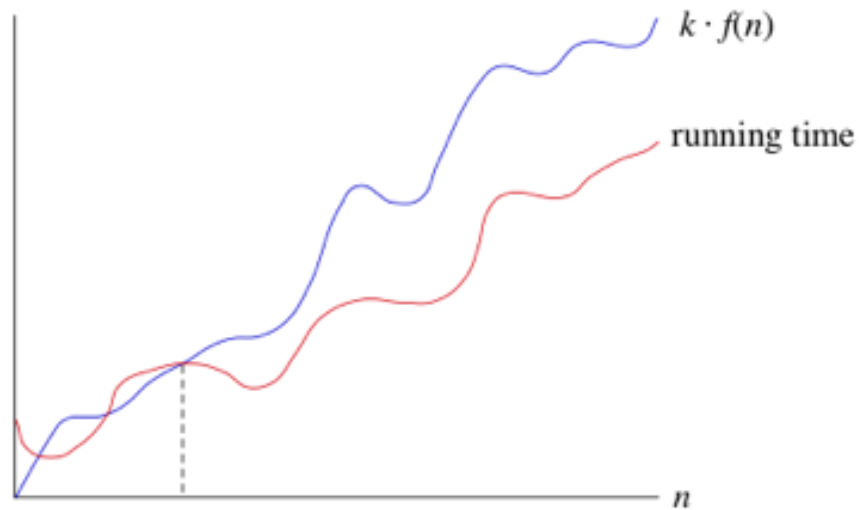
Big O Notation – Formal Definition

Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

$$|f(x)| \leq C|g(x)|$$

whenever $x > k$. [This is read as “ $f(x)$ is big-oh of $g(x)$.”]

Big-O



<https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Asymptotic Notation

- if $f(n)$ is $O(g(n))$ this means that $f(n)$ grows asymptotically no faster than $g(n)$

Complexity Analysis

```
for i in range(n):
```

```
    k = 1 + 1
```

```
    for j in range(n):
```

```
        k = 2 + 2
```

```
        for k in range(n):
```

```
            k = 3 + 3
```

Complexity Analysis

```
for i in range(n):  
    k = 1 + 1  
    for j in range(n):  
        k = 2 + 2  
        for k in range(n):  
            k = 3 + 3
```

Complexity Analysis

```
for i in range(n):  
    k = 1 + 1  
    for j in range(n):  
        k = 2 + 2  
        for k in range(n):  
            k = 3 + 3
```

Complexity Analysis

```
i = n
```

```
while i > 1:
```

```
    <doing something>
```

```
    i = i // 2
```

Complexity Analysis

```
i = 1
```

```
while i <= n:
```

```
    <doing something>
```

```
    i = i * 2
```

Complexity Analysis

```
i = 1
```

```
while i <= n:
```

```
    <doing something>
```

```
    i = i * 5
```

Complexity Analysis

```
for i in range(n):  
    j = 1  
    while j <= n:  
        total += 1  
        j = j * 2
```

Complexity Analysis

```
for i in range(n):  
    for j in range(1,n,2)  
        binarSearch (preSortedArray, j)
```

```
while counter < n:  
    if n % 2 == 0:  
        BinarySearch(preSortedList, counter)  
    else:  
        LinearSearch(preSortedList, counter)  
    counter += 1
```

- You will go with the worst if/else case.

```
while(counter < n) ←  $O(N)$  ... (increases by constant amount)
{
    if(n%2 == 0)
    {
        binarySearch(preSortedArray, counter);
    }
    else
    {
        bruteForceSearch (preSortedArray, counter);
    }
    counter += 1;
}
```

$O(\log(N))$

$O(N)$

Use Big-O Rule 1

Always have to go with the worst if/else case, so $O(N^2)$.

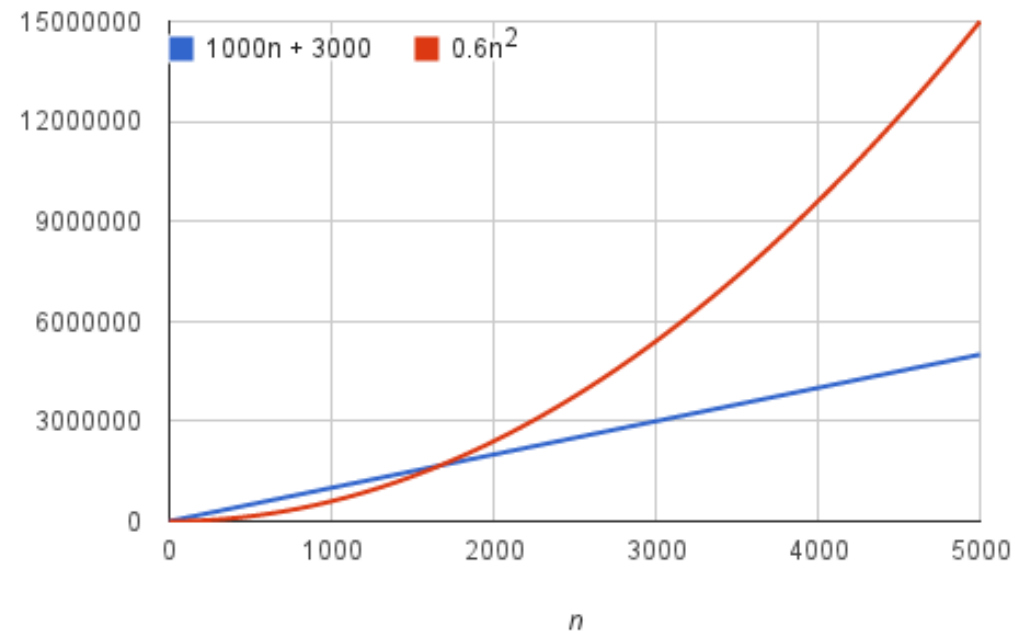
- $O(\log(n)^2)$

```
int counter = 1;
while(counter < n)
{
    binarySearch(preSortedArray, counter);
    counter *= 2;
}
```

```
def f(L,item):  
    N = len(L)  
    S = []  
    for i in range(N**2):  
        S.append(L[i % N])  
    sequentialSearch(S,item)
```

Asymptotic Analysis

- It doesn't really matter what coefficients we use; as long as the running time is $an^2 + bn + c$ for some numbers a , b , and c , there will always be a value of n for which an^2 is greater than $bn + c$ and this difference increases as n increases.



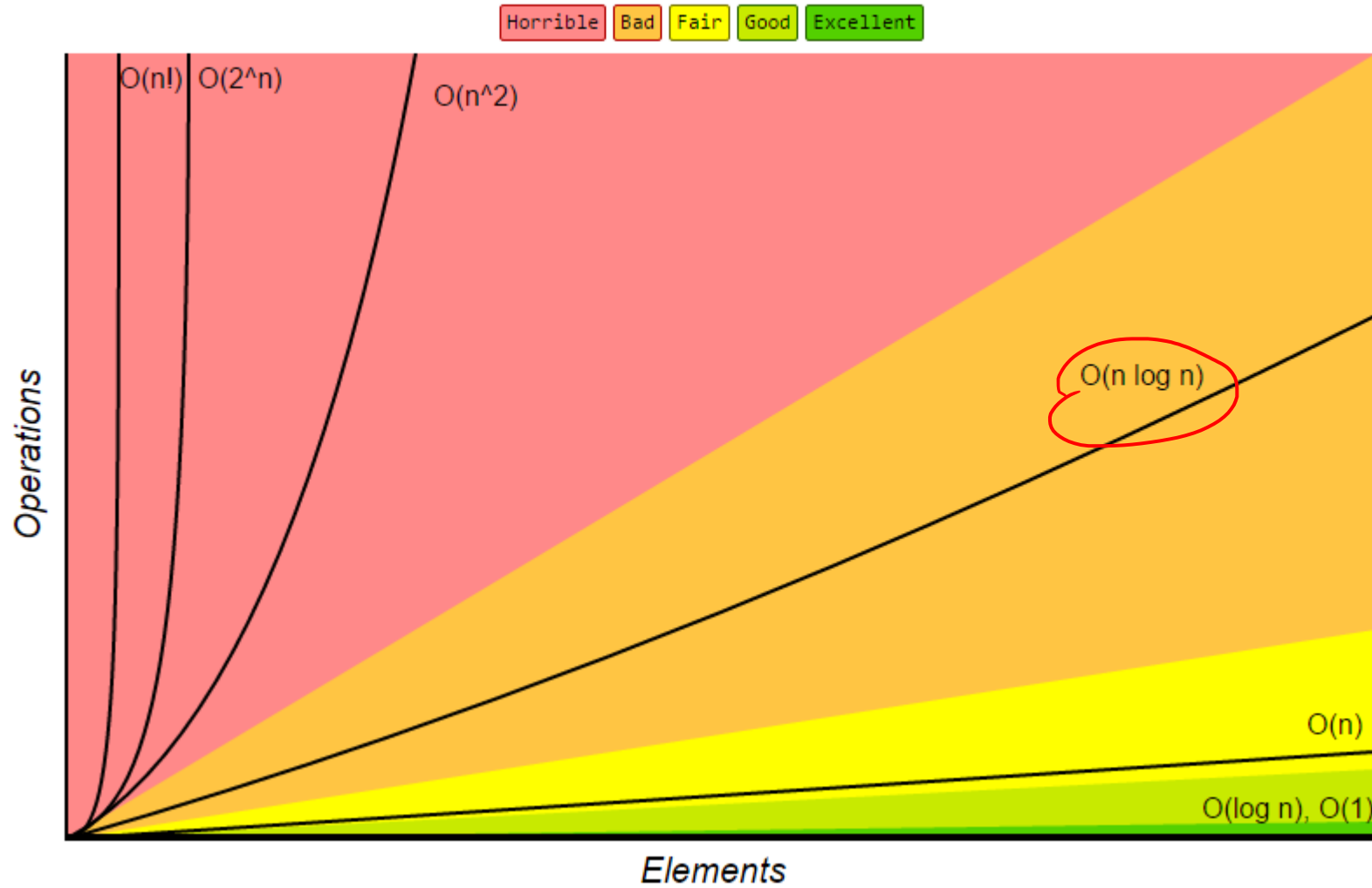
<https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Asymptotic Analysis

- By dropping the less significant terms and the constant coefficients, we can focus on the important part of an algorithm's running time, — its rate of growth. When we drop the constant coefficients and the less significant terms, we use **asymptotic notation**.

1. $\Theta(1)$
2. $\Theta(\lg n)$
3. $\Theta(n)$
4. $\Theta(n \lg n)$
5. $\Theta(n^2)$
6. $\Theta(n^2 \lg n)$
7. $\Theta(n^3)$
8. $\Theta(2^n)$

Big-O Complexity Chart



<http://bigocheatsheet.com/>

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Thanks